

1. 프로젝트 개요

본 프로젝트는 전북대학교 리눅스 프로그래밍 수업에서 진행된 팀 프로젝트로, Linux 환경에서 제공하는 소켓 통신, 파일 시스템, 프로세스, 시스템 콜을 직접 활용하여 C 언어 기반의 다중 사용자 게시판 서버/클라이언트 시스템을 구현하는 것을 목표로 합니다.

서버는 다수의 클라이언트로부터 동시에 접속 요청을 수신하며, 회원가입 및 로그인과 같은 인증 기능을 기반으로 게시글과 댓글을 파일 기반 데이터베이스로 관리합니다.

특히 본 프로젝트는 단순한 콘솔 프로그램을 넘어,

- TCP 소켓을 이용한 클라이언트-서버 구조
- 다중 클라이언트 환경에서의 동시성 처리
- 파일 I/O 및 파일 잠금(lock)을 통한 데이터 무결성 보장
- 시스템 콜 기반의 저수준 구현

을 중심으로, 리눅스 시스템 프로그래밍의 핵심 개념을 실제로 적용하는 데 초점을 맞추었습니다.

2. 기능 요약

본 게시판 시스템은 인증, 게시글, 댓글의 세 영역으로 구성됩니다.

인증 기능

- 회원가입
 - 새로운 사용자 계정을 생성하고 사용자 정보를 파일에 저장
- 로그인
 - 아이디 및 비밀번호 검증을 통해 사용자 인증 수행
 - 서버 세션에서 로그인 상태 유지

게시글 기능

- 게시글 작성
 - 로그인한 사용자만 게시글 작성 가능
 - 작성자 정보, 작성 시각, 내용 등을 파일에 저장
- 게시글 수정
 - 작성자 본인만 게시글 수정 가능
- 게시글 삭제
 - 작성자 권한 검증 후 삭제 처리
- 게시글 검색
 - 제목 또는 작성자 기준으로 게시글 검색 지원

댓글 기능

- 댓글 작성
 - 특정 게시글에 댓글 추가
 - 작성자 및 작성 시각 정보 저장

3. 사전 요구사항

본 프로젝트는 Linux 환경을 기준으로 개발되었습니다. 프로젝트를 빌드 및 실행하기 위해서는 다음 도구들이 필요합니다.

필수 도구

- **gcc** : C 컴파일러
- **make** : Makefile 기반 빌드 도구
- **git** : 소스 코드 다운로드

Ubuntu / Debian 계열

```
sudo apt update
sudo apt install -y gcc make git
```

CentOS / Rocky Linux

```
sudo dnf install -y gcc make git
```

macOS (Homebrew 사용)

```
brew install gcc make git
```

설치가 완료되면 다음 명령어로 정상 설치 여부를 확인할 수 있습니다.

```
gcc --version
make --version
git --version
```

4. 실행 순서

본 프로젝트를 실행하기 위한 단계는 다음과 같습니다.

4.1 저장소 클론

GitHub에서 프로젝트 저장소를 로컬 환경으로 클론합니다.

```
git clone https://github.com/boseung2/linux-board.git
cd linux-board
```

4.2 프로젝트 빌드

프로젝트의 서버와 클라이언트를 빌드합니다. **Makefile**이 제공되므로 **make** 명령어를 사용합니다.

```
make clean
make
```

- **make clean**: 이전에 빌드된 실행 파일 및 오브젝트 파일을 제거합니다.
- **make**: 서버 (**server/server**) 및 클라이언트 (**client/client**) 실행 파일을 빌드합니다.

성공적으로 빌드되면 **server/server**와 **client/client** 실행 파일이 각 디렉터리에 생성됩니다.

4.3 서버 실행

클라이언트가 접속할 수 있도록 서버를 먼저 실행해야 합니다.

```
./server/server
```

서버가 성공적으로 시작되면 다음과 같은 메시지가 출력됩니다.

```
Server is running on port 9000
```

4.4 클라이언트 실행

서버가 실행 중인 상태에서 클라이언트를 실행하여 게시판 시스템에 접속합니다.

```
./client/client
```

클라이언트가 서버에 성공적으로 연결되면 다음과 같은 메시지가 출력됩니다.

```
Connected to server 127.0.0.1:9000
```

이제 클라이언트 콘솔을 통해 게시판 기능을 이용할 수 있습니다.

5. 프로젝트 구조

본 프로젝트의 주요 디렉터리 및 파일 구조는 다음과 같습니다.

```
.
├── Makefile          # 프로젝트 빌드 자동화를 위한 make 파일
├── README.md         # 프로젝트 설명 문서
└── client/           # 클라이언트 관련 소스 코드
```

		client.h	# 클라이언트 공통 헤더		
		main.c	# 클라이언트 메인 함수 (초기화 및 UI 흐름 제어)		
		socket.c	# 서버와의 소켓 통신 처리		
		socket.h	# 소켓 통신 관련 헤더		
		ui_auth.c	# 인증(회원가입, 로그인) UI 및 로직		
		ui_auth.h	# 인증 UI 헤더		
		ui_board.c	# 게시판(글 목록, 상세, 작성, 수정, 삭제, 댓글) UI 및 로직		
		ui_board.h	# 게시판 UI 헤더		
		ui_main_menu.c	# 메인 메뉴 UI 및 로직		
		ui_main_menu.h	# 메인 메뉴 UI 헤더		
		data/	# 서버 데이터 저장 (게시글, 사용자 정보 등)		
		server/	# 서버 관련 소스 코드		
				auth.c	# 사용자 인증 처리 (세션 관리)
				auth.h	# 인증 관련 헤더
				board_controller.c	# 클라이언트 요청(게시판 관련) 처리 및 서비스 호출
				board_controller.h	# 게시판 컨트롤러 헤더
				board_service.c	# 게시글/댓글 데이터 파일 CRUD 로직
				board_service.h	# 게시판 서비스 헤더
				board.h	# 게시글 및 댓글 구조체 정의
				lock.h	# 파일 잠금(fcntl) 관련 헤더
				log.c	# 서버 로깅 기능
				log.h	# 로깅 헤더
				server.c	# 서버 메인 함수 (소켓 초기화, 다중 클라이언트 관리)
				user.c	# 사용자 정보(회원가입, 로그인) 파일 CRUD 로직
				user.h	# 사용자 정보 관련 헤더

주요 디렉터리 설명

- **client/**: 사용자 인터페이스 및 서버와의 통신 로직을 담당하는 클라이언트 측 코드입니다. 사용자의 입력을 받고 서버에 요청을 전송하며, 서버 응답을 파싱하여 사용자에게 보여주는 역할을 합니다.
- **server/**: 클라이언트의 요청을 처리하고 데이터를 관리하는 서버 측 코드입니다. 소켓 통신, 사용자 인증, 게시판 데이터 관리, 파일 잠금 등을 처리합니다.
- **data/**: 서버가 관리하는 사용자 계정 정보, 게시글 및 댓글 데이터가 이진 파일 형태로 저장되는 디렉터리입니다. 서버 실행 시 자동으로 생성됩니다.

6. 기능 상세 정보 (스크린샷과 함께)

각 주요 기능에 대한 상세 설명 및 스크린샷입니다.

6.1 인증 기능

6.1.1 회원가입

새로운 사용자 계정을 생성합니다. 아이디와 비밀번호를 입력받아 서버에 전송하고, 중복 확인 후 계정을 생성합니다.

```
=== 메인 메뉴 ===
1. 로그인
2. 회원가입
3. 종료
선택 : 2

=== 회원가입 ===
ID: testuser
PW: testuser
[SERVER] OK SIGNUP
회원가입이 완료되었습니다. 메인 메뉴로 돌아갑니다.

계속하려면 Enter 키를 누르세요 ...
```

6.1.2 로그인

등록된 아이디와 비밀번호로 시스템에 로그인합니다. 성공 시 사용자 세션이 유지되며, 게시판 기능에 접근할 수 있습니다.

```
=== 메인 메뉴 ===
1. 로그인
2. 회원가입
3. 종료
선택 : 1

=== 로그인 ===
ID: testuser
PW: testuser
[SERVER] OK LOGIN
로그인 성공!

계속하려면 Enter 키를 누르세요 ...
```

6.2 게시글 기능

6.2.1 게시글 목록 조회

작성된 모든 게시글을 최신순으로 조회합니다. 페이지네이션을 지원하며, 제목 또는 작성자로 검색할 수 있습니다.

```
===== 글 목록 (페이지 : 1) =====
번호 | 제목 | 작성자 | 작성일
-----+-----+-----+-----
31 | test | testuser | 2025-12-14 20:23:32
30 | REST API 설계 모범 사례 | nest_master | 2025-12-08 10:02:25
29 | 확장 가능한 시스템을 설계하 ... | spring_lover | 2025-11-23 09:02:21
28 | 클라우드 제공업체 선택: AWS... | db_master | 2025-11-15 13:35:16
27 | 오픈 소스 프로젝트에 기여하 ... | pythonista | 2025-10-27 11:57:54
26 | 트래픽 많은 데이터베이스를 ... | db_master | 2025-10-14 11:22:48
25 | 머신 러닝 시작하기 | db_master | 2025-09-16 14:11:03
24 | SQL vs NoSQL: 언제 무엇을 ... | db_master | 2025-08-26 11:29:51
23 | 애자일 vs 폭포수: 어떤 소프 ... | lia | 2025-08-12 15:09:51
22 | 서버리스 컴퓨팅의 부상 | nest_master | 2025-07-27 13:18:01
-----+-----+-----+-----

선택 :
[글번호] → 해당 글 상세보기
[P] 이전 페이지 [N] 다음 페이지
[S] 검색
[C] 글 작성
[R] 새로고침 (검색 초기화)
[B] 뒤로 (게시판 메인으로)
입력 : █
```

```
===== 글 목록 (페이지 : 1) =====
번호 | 제목 | 작성자 | 작성일
-----+-----+-----+-----
31 | test | testuser | 2025-12-14 20:23:32
30 | REST API 설계 모범 사례 | nest_master | 2025-12-08 10:02:25
29 | 확장 가능한 시스템을 설계하 ... | spring_lover | 2025-11-23 09:02:21
28 | 클라우드 제공업체 선택: AWS... | db_master | 2025-11-15 13:35:16
27 | 오픈 소스 프로젝트에 기여하 ... | pythonista | 2025-10-27 11:57:54
26 | 트래픽 많은 데이터베이스를 ... | db_master | 2025-10-14 11:22:48
25 | 머신 러닝 시작하기 | db_master | 2025-09-16 14:11:03
24 | SQL vs NoSQL: 언제 무엇을 ... | db_master | 2025-08-26 11:29:51
23 | 애자일 vs 폭포수: 어떤 소프 ... | lia | 2025-08-12 15:09:51
22 | 서버리스 컴퓨팅의 부상 | nest_master | 2025-07-27 13:18:01
-----+-----+-----+-----

선택 :
[글번호] → 해당 글 상세보기
[P] 이전 페이지 [N] 다음 페이지
[S] 검색
[C] 글 작성
[R] 새로고침 (검색 초기화)
[B] 뒤로 (게시판 메인으로)
입력 : s
검색 옵션 :
1. 제목으로 검색
2. 작성자로 검색
선택 : 1
검색어 : 개발█
```

===== 글 목록 (페이지 : 1) =====

>> 검색 모드 (제목 : '개발') <<

번호	제목	작성자	작성일
23	애자일 vs 폭포수 : 어떤 소프...	lia	2025-08-12 15:09:51
10	프론트엔드 개발의 미래	js_ninja	2025-03-24 13:36:09

선택 :

[글번호] → 해당 글 상세보기

[P] 이전 페이지 [N] 다음 페이지

[S] 검색

[C] 글 작성

[R] 새로고침 (검색 초기화)

[B] 뒤로 (게시판 메인으로)

입력 : █

===== 글 목록 (페이지 : 1) =====

번호	제목	작성자	작성일
31	test	testuser	2025-12-14 20:23:32
30	REST API 설계 모범 사례	nest_master	2025-12-08 10:02:25
29	확장 가능한 시스템을 설계하...	spring_lover	2025-11-23 09:02:21
28	클라우드 제공업체 선택 : AWS...	db_master	2025-11-15 13:35:16
27	오픈 소스 프로젝트에 기여하...	pythonista	2025-10-27 11:57:54
26	트래픽 많은 데이터베이스를 ...	db_master	2025-10-14 11:22:48
25	머신 러닝 시작하기	db_master	2025-09-16 14:11:03
24	SQL vs NoSQL: 언제 무엇을 ...	db_master	2025-08-26 11:29:51
23	애자일 vs 폭포수 : 어떤 소프...	lia	2025-08-12 15:09:51
22	서버리스 컴퓨팅의 부상	nest_master	2025-07-27 13:18:01

선택 :

[글번호] → 해당 글 상세보기

[P] 이전 페이지 [N] 다음 페이지

[S] 검색

[C] 글 작성

[R] 새로고침 (검색 초기화)

[B] 뒤로 (게시판 메인으로)

입력 : s

검색 옵션 :

1. 제목으로 검색

2. 작성자로 검색

선택 : 2

검색어 : db_master█

===== 글 목록 (페이지 : 1) =====

>> 검색 모드 (작성자 : 'db_master') <<

번호	제목	작성자	작성일
28	클라우드 제공업체 선택 : AWS...	db_master	2025-11-15 13:35:16
26	트래픽 많은 데이터베이스를 ...	db_master	2025-10-14 11:22:48
25	머신 러닝 시작하기	db_master	2025-09-16 14:11:03
24	SQL vs NoSQL: 언제 무엇을 ...	db_master	2025-08-26 11:29:51

선택 :

[글번호] → 해당 글 상세보기

[P] 이전 페이지 [N] 다음 페이지

[S] 검색

[C] 글 작성

[R] 새로고침 (검색 초기화)

[B] 뒤로 (게시판 메인으로)

입력 : █

6.2.2 게시물 상세 조회

특정 게시물의 제목, 작성자, 내용, 작성일시 등을 상세하게 조회합니다.

```
===== 글 상세 =====
글 번호 : 30
제목 : REST API 설계 모범 사례
작성자 : nest_master
작성일 : 2025-12-08 10:02:25
-----
내용 :
깔끔하고 유지보수하기 좋은 REST API를 설계하기 위한 모범 사
례에는 어떤 것들이 있을까요? URL 구조, 버전 관리, 상태 코드,
오류 처리 등에 대해 이야기하고 싶습니다.
-----

===== 댓글 (1) =====
[spring_lover] 리소스 URL에는 명사를 사용하세요 (예 : /users,
/posts). 그리고 HTTP 동사 (GET, POST, PUT, DEL
ETE)를 사용하여 리소스를 조작하세요. 항상 적
절한 HTTP 상태 코드를 사용하여 요청 결과를 나
타내야 합니다. (2025-12-08 10:11)
=====

1. 댓글 작성
2. 글 수정
3. 글 삭제
4. 목록으로 돌아가기
선택 : █
```

6.2.3 게시물 작성

로그인한 사용자는 새로운 게시물을 작성할 수 있습니다. 제목과 내용을 입력하여 게시물을 등록합니다.

```
===== 글 작성 =====
제목 : 테스트 제목 입니다.
내용 (입력 후 .만 단독으로 입력하면 종료):

> 테스트 내용 입니다.
> 테스트 내용 두 번째 줄 입니다.
> .
-----
1. 등록
2. 취소
선택 : 1
[완료] 32번 게시물이 등록되었습니다.

계속하려면 Enter 키를 누르세요 ...█
```

6.2.4 게시물 수정

작성자 본인만 게시물을 수정할 수 있습니다. 기존 내용을 바탕으로 제목과 내용을 변경합니다.


```

===== 글 수정 =====
기존 제목 : 테스트 제목 입니다 .
새 제목 (Enter 시 기존 제목 유지) : 수정된 제목 입니다 .
기존 내용 :
테스트 내용 입니다 .
테스트 내용 두 번째 줄 입니다 .

새 내용 (입력 후 .만 단독으로 입력하면 종료) :

> 수정된 내용 입니다 .
> 수정이 정상적으로 됩니다 .
> .
[완료] 글이 수정되었습니다 .

계속하려면 Enter 키를 누르세요 ...

```

6.2.5 게시물 삭제

작성자 본인만 게시글을 삭제할 수 있습니다. 게시글은 실제로 삭제되지 않고 `is_deleted` 플래그를 통해 비활성화됩니다 (Soft Delete).

```

===== 글 상세 =====
글 번호 : 32
제목 : 수정된 제목 입니다 .
작성자 : testuser
작성일 : 2025-12-14 20:30:32
-----
내용 :
수정된 내용 입니다 .
수정이 정상적으로 됩니다 .

-----

===== 댓글 (2) =====
[testuser] 테스트 댓글 입니다 . (2025-12-14 20:38)
[testuser] 테스트 댓글 추가 합니다 . (2025-12-14 20:38)
=====

1. 댓글 작성
2. 글 수정
3. 글 삭제
4. 목록으로 돌아가기
선택 : 3
글 삭제를 위해 비밀번호를 입력하세요 : testuser
정말 이 글을 삭제하시겠습니까? (Y/N) : y
[완료] 글이 삭제되었습니다 .

계속하려면 Enter 키를 누르세요 ...

```

6.3 댓글 기능

6.3.1 댓글 작성

게시글 상세 화면에서 댓글을 작성할 수 있습니다. 작성된 댓글은 게시글 하단에 표시됩니다.

```
===== 글 상세 =====
글 번호 : 32
제목 : 수정된 제목 입니다 .
작성자 : testuser
작성일 : 2025-12-14 20:30:32
-----
내용 :
수정된 내용 입니다 .
수정이 정상적으로 됩니다 .

-----

===== 댓글 (1) =====
[testuser] 테스트 댓글 입니다 . (2025-12-14 20:38)
=====

1. 댓글 작성
2. 글 수정
3. 글 삭제
4. 목록으로 돌아가기
선택 : 1
댓글 내용 : 테스트 댓글 추가 합니다 .█
```

7. 설계 및 구현 시 중점 고려 사항

7.1 로그인 및 세션 관리 시스템

- 로그인 성공 시 서버는 소켓(fd)과 사용자 ID를 세션으로 매핑하여 관리합니다.
- 이후 요청에서는 클라이언트로부터 사용자 ID를 직접 전달받지 않고, 서버 세션을 통해 인증 상태를 유지합니다.

7.2 파일 잠금 기반 동시성 제어

- 다수의 클라이언트가 동시에 파일에 접근하는 환경을 고려하여 `fcntl` 기반 파일 잠금을 적용하였습니다.
- 이를 통해 게시글 및 사용자 데이터 파일 접근 시 Race Condition을 방지합니다.

7.3 자동 로그아웃 시스템

- 5분 동안 활동이 없는 세션은 자동으로 로그아웃 처리됩니다.
- 이를 통해 서버 자원 점유를 최소화하고 안정적인 운영을 고려하였습니다.

7.4 파일 기반 데이터 관리 구조

- 별도의 데이터베이스를 사용하지 않고, 파일 시스템을 직접 활용하여 데이터를 관리합니다.
- 서버 재시작 이후에도 기존 데이터가 유지되도록 설계하였습니다.

7.5 서버 로깅 및 디버깅 지원

- 서버의 주요 동작(접속, 인증, 파일 접근 등)을 로그로 기록하여 디버깅과 시연에 활용합니다.

7.6 권한 검증 시스템

- 게시글 수정 및 삭제 시 서버 세션 정보와 게시글 작성자를 비교하여 권한을 검증합니다.
- 클라이언트 입력에 의존하지 않는 서버 중심 권한 판단 구조를 적용하였습니다.

7.7 다중 클라이언트 처리 구조

- 서버는 다수의 클라이언트 접속을 처리할 수 있도록 소켓 기반 구조로 설계되었습니다.
- 세션 관리와 파일 잠금을 통해 동시 접속 환경에서도 안정적인 동작을 보장합니다.

7.8 게시글 최대 개수 제한 설계

- 파일 크기 무한 증가를 방지하기 위해 게시글 최대 개수를 제한하였습니다.
- 제한 초과 시 게시글 생성 요청을 거부하여 서버 안정성을 고려하였습니다.

8. 역할 분담

정보승 (@boseung2)

서버-클라이언트 소켓 통신 구조를 설계하고 구현하였으며, 회원가입 및 로그인 기능을 담당하여 사용자 인증 흐름을 완성하였다. 또한 게시판의 사용성을 향상시키기 위한 부가 기능을 중심으로 구현하였다.

주요 기능 구현

- 서버-클라이언트 소켓 통신 구조 설계 및 구현
- 회원가입 및 로그인 기능 구현
- 자동 로그아웃 기능 구현
- 게시글 목록 페이지네이션 기능 구현 (이전 페이지 / 다음 페이지)
- 게시글 목록 검색 기능 구현 (제목, 작성자 기준)
- 시연 영상 제작

이인애 (@lia0519)

게시판 시스템의 핵심 기능(CRUD)을 담당하였으며, 다중 클라이언트 환경에서의 안정적인 동작을 위해 동시성 제어와 권한 관리를 중점적으로 구현하였다.

주요 기능 구현

- 게시글 기능 구현 (Create, Read, Update, Delete)
- 게시판 파일에 대한 전체 구조 단위 Lock 적용
- 작성자 본인만 게시글 수정/삭제가 가능하도록 권한 검사 로직 구현
- 게시글 상세 보기에서 내용이 한 줄만 출력되던 오류 수정
- 발표 영상 제작